

Passage à OpenGL (ES) 3.x

malek.bengougam@gmail.com

Partie 1 – entrées et sorties des shaders

Afin d'effectuer le même rendu mais en simulant un monde plus large il nous faut une matrice pour représenter une caméra. (cf. TP camera)

Partie 2 – UBO

Rappel sur le passage des matrices

Afin d'effectuer le même rendu mais en simulant un monde plus large il nous faut une matrice de projection (orthographique ou perspective) ainsi qu'une matrice pour simuler la caméra, et bien entendu une matrice pour placer et orienter l'objet dans le monde.

La matrice de projection a pour rôle de transformer les coordonnées exprimées dans le repère du monde (le plus souvent relativement au point de vue de la caméra) vers les coordonnées normalisées.

Ces matrices sont communes pour l'ensemble des vertices de l'objet, on les passera donc en tant que variable uniforme. Les variables uniformes ont aussi un « location » mais il n'est pas possible d'utiliser `layout(location)` pour spécifier cet identifiant.

On est donc obligé de continuer à utiliser la fonction `glGetUniformLocation()` pour récupérer cette information que l'on devra passer aux fonctions `glUniform***()` qui permettent de mettre à jour les données côté GPU.

Basic.vs

```
#version 330
```

```
layout(location=0) in vec4 a_position;
```

```
uniform mat4 u_worldMatrix;
```

```
uniform mat4 u_viewMatrix;
```

```
uniform mat4 u_projectionMatrix;
```

```
void main(void)
```

```
{
```

```
    gl_Position = u_projectionMatrix * u_viewMatrix * u_worldMatrix * a_position;
```

```
}
```

Côté C++ on va stocker nos matrices projection et vue dans une structure ViewProj et la matrice monde dans une structure Object.

```
struct ViewProj
```

```
{
```

```
    mat4 viewMatrix;
```

```
    mat4 projectionMatrix;
```

```
} g_Camera;
```

```
struct Object
```

```
{
```

```
    mat4 worldMatrix;
```

```
} g_Obj;
```

Rappel : la fonction **glUniformMatrix4fv()** nécessite l'adresse mémoire de la matrice.

```
// [...]
g_Camera.projectionMatrix = perspectiveFov(45.f, (float)width, (float)height, 0.1f,
1000.f);

vec4 position = vec4(0.0f, 0.0f, 2.0f, 1.0f);
g_Camera.viewMatrix = lookAt(vec3(position), vec3(0.f), vec3(0.f, 1.f, 0.f));

auto projLocation = glGetUniformLocation(basicProgram, "u_perspectiveMatrix");
glUniformMatrix4fv(projLocation, 1, GL_FALSE, g_Camera.projectionMatrix);

auto viewLocation = glGetUniformLocation(basicProgram, "u_viewMatrix");
glUniformMatrix4fv(viewLocation, 1, GL_FALSE, g_Camera.viewMatrix);

auto worldLocation = glGetUniformLocation(basicProgram, "u_worldMatrix");
glUniformMatrix4fv(worldLocation, 1, GL_FALSE, g_Objets.worldMatrix);
// [...]
```

Uniform Buffer Objects (UBO)

On a vu comment réduire la charge sur la bande passante pour les attributs avec les VBO ainsi que le nombre de commande OpenGL avec le VAO. Cependant on se retrouve toujours à envoyer à chaque frame, qui plus est séparément, les matrices de projections et de vues alors qu'elles sont censées être constantes pour l'ensemble d'une trame.

Autrement dit, si l'on dessine plusieurs objets ces derniers vont utiliser ces mêmes matrices.

C'est pourquoi depuis OpenGL 3 on a la possibilité de partager les données uniformes entre plusieurs shaders via les **Uniform Buffer Objects (UBO)**.

Le principe de base reste similaire à celui des VBO ou autres BO, on va là encore utiliser les fonctions `glGenBuffers()`, `glDeleteBuffers()`, `glBindBuffer()` et `glBufferData()` pour manager notre UBO.

La différence réside en pratique dans le type de BO qui devient **GL_UNIFORM_BUFFER** et dans la façon dont on va rendre accessible aux shaders nos UBO.

Premièrement il nous faut un `GLuint` afin de stocker l'identifiant du BO et ensuite on va simplement allouer de la mémoire pour stocker nos deux matrices (à l'initialisation).

```
glGenBuffers(1, &g_Camera.UBO);
glBindBuffer(GL_UNIFORM_BUFFER, g_Camera.UBO);
// notez le GL_STREAM_DRAW pour indiquer que les données changent peu par frame
glBufferData(GL_UNIFORM_BUFFER, sizeof(mat4) * 2, NULL, GL_STREAM_DRAW);

glBindBufferBase(GL_UNIFORM_BUFFER, bindingPoint, g_Camera.UBO);
glBindBuffer(GL_UNIFORM_BUFFER, 0);
```

La fonction **glBindBufferBase()** permet d'indiquer à OpenGL sur quel point d'ancrage (*binding point*) il sera possible de lire ces données partagées.

La variable **bindingPoint** contient le numéro (un entier) d'un point d'ancrage virtuel qui va nous permettre de faire le lien entre l'UBO et l'*uniform block* dans le shader.

Qu'est-ce qu'un **uniform block** ? C'est une espèce de structure qui regroupe en un bloc (un bloc étant entre simplement un « scope » de data entre accolades), un ensemble de variables uniformes. Dans le Vertex Shader `basic.vs` on va donc maintenant avoir le code suivant :

```
uniform mat4 u_worldMatrix;
layout(std140) uniform ViewProj
{
    mat4 u_viewMatrix;
    mat4 u_projectionMatrix;
};
```

layout(std140) n'est pas obligatoire dans notre cas et sert juste à indiquer comment les données sont alignées en mémoire (alignement de la structure, taille du bourrage etc...). En l'occurrence, l'alignement par défaut ici force un empaiquetage et un alignement sur 16 octets.

Attention ! Si vous utilisez des tableaux (de floats par exemple) chaque float sera espacé de 16 octets ! La spécification std430 permet de réduire cet espacement, mais n'est pas forcément supporté avec les UBO/Uniform Block (cf. SSBO/buffer que nous verrons ultérieurement).

Il faut aussi spécifier à OpenGL comment lier un shader avec le point d'ancrage virtuel. Cela se fait à l'aide de ces deux fonctions :

```
auto blockIndex = glGetUniformLocation(program, "ViewProj");
glUniformBlockBinding(program, blockIndex, blockBinding);
```

La première récupère l'équivalent du location de l'uniform block tandis que la seconde effectue concrètement le lien (toujours à l'initialisation).

Note : en OpenGL 4.2+, ou bien avec l'extension ARB_shading_language_420pack, on peut utiliser le « layout (binding = ?) » sur un Uniform Block. Dans ce cas, il n'est plus nécessaire d'avoir recours aux deux fonctions précédentes.

A partir de maintenant, toutes les modifications sur l'UBO seront automatiquement répercutées au niveau des shaders. Plus besoin d'update ces variables uniformes dans chacun des shaders.

Le code de mise à jour des matrices devient alors le suivant :

```
g_Camera.projectionMatrix = ...
g_Camera.viewMatrix = ...

glBindBuffer(GL_UNIFORM_BUFFER, g_Camera.UBO);
glBufferData(GL_UNIFORM_BUFFER, sizeof(mat4)*2, g_Camera.viewMatrix, GL_STREAM_DRAW);
// on peut aussi utiliser glBufferSubData, pas forcément plus optimal
//glBufferSubData(GL_UNIFORM_BUFFER, 0, sizeof(glm::mat4) * 2,
glm::value_ptr(g_Camera.viewMatrix));
auto worldLocation = glGetUniformLocation(basicProgram, "u_worldMatrix");
glUniformMatrix4fv(worldLocation, 1, GL_FALSE, g_Obj.worldMatrix);
```

Partie 3 – Textures

Le principal changement au sujet des textures concerne les fonctions GLSL `texture2D()`, `textureCube()` etc.. A partir d'OpenGL 3, tous les samplers peuvent être échantillonnés avec la fonction générique **texture()**.

Note : A partir d'OpenGL 4.2 il est aussi possible d'utiliser **layout(binding=)** pour un sampler. Ce qui supprime l'utilité de faire appel à `glUniform1i` côté code client.

Le concept de Texture Object pouvait dans certains cas être problématique. En effet, le fait qu'un Texture Object contienne à la fois les données de la texture et les états/paramètres de cette dernière

implique que le driver doit tout reconfigurer à chaque changement de Texture Object –via **glBindTexture()**.

La plupart du temps, si l'on souhaite effectivement changer de visuel, les développeurs 3D ont tendance à utiliser plutôt les mêmes paramètres (filtres, répétitions, lods...).

De plus l'image elle-même une fois qu'elle a été chargée est rarement modifiée en temps-réel –que cela soit en taille ou bien en termes des données contenues.

Le cas particulier étant d'abord celui du chargement des niveaux de mipmap et comme on l'a vu, OpenGL doit s'assurer que la texture est valide en vérifiant au préalable la bonne spécification des textures (niveaux de détail, format des pixels etc..), et très souvent ceci se fait au moment du **glBindTexture()**, donc potentiellement à chaque utilisation de la texture.

Ces deux constats ont permis l'introduction de deux nouveaux concepts : les textures immuables et les Sampler Objects.

Une texture immuable (immutable texture) est un Texture Object simplifié pour lequel il n'est avant tout pas possible de modifier sa dimension et son format interne une fois que ceux-ci ont été spécifiés. Ceci a comme vertu d'optimiser initialement les allocations mémoires et réduire le temps perdu en validation.

En OpenGL 3.3+ la fonction **glTexStorage2D** est préférable à **glTexImage2D**. En effet **glTexImage2D** reste une fonction générique qui permet de réallouer/reconfigurer une texture ce qui est très coûteux. **Les textures immuables (Immutable Texture)** ont des paramètres fixes et sont donc traitées plus rapidement par le pilote. Exemple :

```
// Allocation d'une texture, le deuxième paramètre indique le nombre de mipmap
glTexStorage2D(GL_TEXTURE_2D, 1, GL_RGBA8, w, h);
// remplissage
glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, 0, w, h, GL_RGBA, GL_UNSIGNED_BYTE, data);
```

Les Sampler Objects ont pour but de remplacer l'usage de **glTexParameter()** et ses variantes. L'idée est ici de créer un objet GL contenant ces valeurs de filtres, répétitions etc... et de ne changer de Sampler Object que lorsque l'on en a vraiment besoin.

Ceci est très important pour le rendu car le driver perd moins de temps en configuration.

Le Sampler Object étant un objet GL, on retrouve donc la panoplie usuelle de fonctions de gestion.

glGenSamplers() permet de créer les objets, **glDeleteSamplers()** de les supprimer.

On active un sampler via la fonction **glBindSampler(GLenum unit, GLuint samplerObject)** où « unit » correspond à l'unité de texture qui va être configurée (**GL_TEXTURE0**, **GL_TEXTURE1**...) et « samplerObject » correspond à un objet créé via **glGenSamplers()**.

Comme très souvent la valeur d'objet 0 (zéro) indique le rétablissement vers le fonctionnement par défaut (donc désactive l'usage du Sampler Object sur l'unité de texture choisie).

Une fois le Sampler Object actif on peut le reconfigurer via les fonctions **glSamplerParameter()** et variantes qui fonctionnent comme **glTexParameter()**.

Partie 4 – quelques nouveautés du pipeline

Rendu sans attributs

De nouvelles variables prédéfinies ont fait leur apparition :

```
in highp int gl_VertexID;  
in highp int gl_InstanceID;
```

- **gl_VertexID** contient l'index du vertex dans la primitive ou le maillage.
- **gl_InstanceID** contient le numéro de l'appel de rendu, lorsque le rendu est instancié (en utilisant la technique du hardware instancing aussi appelé geometry instancing).

Nous verrons **gl_InstanceID** prochainement en abordant l'instancing.

gl_VertexID permet donc d'identifier une invocation du vertex shader pendant le rendu d'un ensemble de primitive.

On peut récupérer cet identifiant pour n'importe quel vertex ici compris dans un cadre un peu déroutant qui est le rendu sans attributs.

Comme son nom l'indique le rendu sans attributs (attribute-less rendering) permet de rendre des primitives sans passer par les fonctions de définitions d'attributs, donc ni VAO, ni VBO etc... ni appels à **glVertexAttribPointer()** !

L'astuce est la suivante. Il suffit tout simplement de lancer un appel de rendu comme le suivant, qui dessine 2 triangles à l'aide de 4 points suivants la méthode de triangle strip.

```
glDrawArrays( GL_TRIANGLE_STRIP, 0, 4 );
```

Dans le shader, on définit alors un tableau contenant les attributs de chacun des 4 sommets (ici des positions 2D) et on index ensuite ce tableau à l'aide de **gl_VertexID** !

```
#version 330
```

```
const vec2 quad[4] = vec2[] (  
    vec2(-1.0, 1.0),  
    vec2(-1.0, -1.0),  
    vec2( 1.0, 1.0),  
    vec2( 1.0, -1.0)  
);  
  
void main()  
{  
    gl_Position = vec4( quad[ gl_VertexID ], 0.0, 1.0);  
}
```

Geometry shaders

A partir d'OpenGL 3.x un nouveau type d'unité de calcul shader à fait son apparition officielle : le Geometry Shader. Il est toutefois possible d'utiliser un Geometry Shader en OpenGL 2.x si le hardware et le pilote le permettent via l'extension `Ext_Geometry_Shader4` mais la syntaxe est sensiblement différente des Geometry Shaders en OpenGL 3.+.

Tout GPU supportant la norme OpenGL ≥ 3.1 contient un support des Geometry Shaders. Le geometry Shader est par contre optionnel en OpenGL ES 3.1 et inexistant (sauf extension) avant.

Un Geometry Shader s'intercale entre le Vertex Shader et le Fragment Shader.

Il reçoit en entrée un tableau contenant les sorties de tous les Vertex Shaders exécutés pour le traitement d'une primitive.

Ainsi, si la primitive est `GL_POINTS`, le tableau ne contiendra qu'un seul vertex. Dans le cas où la primitive est `GL_LINES` ou `GL_LINE_LIST` ou `GL_LINE_LOOP` il en recevra deux, et trois vertices dans le cas de `GL_TRIANGLES`, `GL_TRIANGLE_STRIP` et `GL_TRIANGLE_FAN`.

On peut spécifier le type de données en entrée et en sortie du Geometry Shader de cette façon

```
layout(triangles) in;  
layout(triangle_strip, max_vertices = 3) out;
```

Dans cet exemple on suppose donc un appel de dessin de type `GL_TRIANGLES`.

La metadata `max_vertices` permet d'indiquer le nombre de vertices que va produire en sortie notre GS. Ceci est important pour le pilote afin qu'il puisse réserver l'espace nécessaire à l'exécution du shader dans la mémoire dédiée au GS du GPU.

PRIMITIVE GL	TYPE EN ENTREE	NOMBRE DE VERTEX
<code>GL_POINTS</code>	points	1
<code>GL_LINES</code>	lines	2
<code>GL_LINE_LIST</code>	lines	2
<code>GL_LINE_STRIP</code>	lines	2
<code>GL_TRIANGLES</code>	triangles	3
<code>GL_TRIANGLE_STRIP</code>	triangles	3
<code>GL_TRIANGLE_FAN</code>	triangles	3

Lorsqu'il est actif le GS prend donc en entrée les sorties (out) du Vertex Shader, en premier lieu desquelles `gl_Position`, et émet en sortie les informations nécessaires au rasterizer qui seront interpolées et envoyées en entrée (in) du Fragment Shader.

Un Geometry Shader ne peut produire que trois types de données en sortie :

- points
- line_strip
- triangle_strip

Pour chaque vertex en sortie il faut au moins affecter une valeur à `gl_Position` et terminer par un appel à la fonction **EmitVertex()** qui indique la fin de la description du vertex. **EndPrimitive()** quant à elle permet optionnellement d'indiquer la fin d'une primitive.

`EndPrimitive()` permet dans le cas de `line_strip` et `triangle_strip` de générer des primitives disjointes.

Exemple d'un `basic.gs` qui ne fait qu'envoyer en sortie les données telles qu'elles ont été reçues:

```
#version 150
```

```
layout(triangles) in;  
layout(triangle_strip, max_vertices = 3) out;
```

```
void main(void)  
{  
    // gl_in.length() retourne 3 car nous avons un triangle en entree  
    for (int i = 0; i < gl_in.length(), i++) {  
        gl_Position = gl_in[i].gl_Position;  
        EmitVertex();  
    }  
    // EndPrimitive(); // optionnel ici  
}
```

Si l'on remplace la ligne

```
layout(triangle_strip, max_vertices = 3) out;  
par  
layout(points, max_vertices = 3) out;
```

le Geometry Shader produira seulement les 3 vertices en entrée sous forme de points isolés (qui pourront d'ailleurs être grossi par l'usage de `glPointSize()` par ex.)

Exemple d'un Geometry Shader produisant des lignes (arêtes) à partir de triangles.

Il s'agit presque du même code à l'exception du premier vertex qui est répété pour former une succession (bande = strip) de 3 lignes à partir de 4 vertex émis : (0,1) (1,2) (2, 0 bis)

```
#version 150
```

```
layout(triangles) in;  
layout(line_strip, max_vertices = 4) out;
```

```
void main(void)  
{  
    for (int i = 0; i < gl_in.length(), i++) {  
        gl_Position = gl_in[i].gl_Position;  
        EmitVertex();  
    }  
    gl_Position = gl_in[0].gl_Position; // 0 bis  
    EmitVertex();  
}
```

Un autre usage très important, mais relativement limité est l'amplification. Cela consiste à créer dynamiquement de nouvelles primitives à partir des données en entrée.

Voir <https://open.gl/geometry> pour un exemple d'amplification à partir d'un point.